



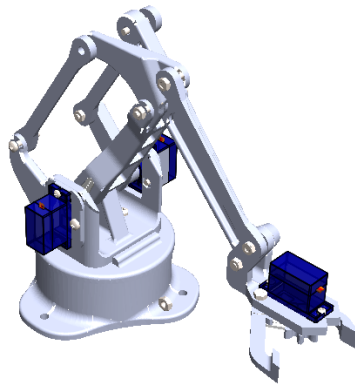
University of Carthage

National Institute of Applied Sciences and Technology

Department : Department of Physics and Instrumentation

Personal Professional Project

**Design and Realization
of a 4-Axis Robotic Arm
with Digital Twin under ROS2**



Prepared by :

Mohamed Amine Benkaid

Rayen Zahzah

Supervisor :

Dr. Boutheina Ben Abdelaziz

Examiner :

Mr. Hosni Saidi

Academic Year 2025 – 2026

Acknowledgements

At the end of this personal professional project, we wish to express our deepest gratitude to all the people who contributed, directly or indirectly, to the success of this work.

We extend our most sincere thanks to our supervisor, **Dr. Boutheina Ben Abdelaziz**, for the quality of her guidance, the relevance of her advice, and the trust she placed in us throughout this project. Her availability and expertise were decisive in orienting our technical choices.

Our thanks also go to the members of the jury, and in particular to **Dr. Hosni Saidi**, who honored us by examining and evaluating this work.

We would like to thank the entire teaching staff of the National Institute of Applied Sciences and Technology (INSAT) for the quality of the education provided, which allowed us to approach this project with the necessary methodological and technical tools.

We finally express our gratitude to our families and fellow students for their constant support and encouragement throughout this experience.

Contents

Acknowledgements

List of Abbreviations

General Introduction	1
1 General Context and Specifications	3
1.1 Introduction	3
Introduction	3
1.2 General Context of Manipulation Robotics	3
1.3 The Digital Twin Concept	4
1.3.1 Definition and Characteristics	4
1.3.2 Application to a Robotic Arm	4
1.4 Specifications	4
1.4.1 Functional Requirements	4
1.4.2 Technical Constraints	5
References	6
2 Mechanical Structure	7
2.1 Introduction	7
Introduction	7
2.2 Mechanical Structure of the 4-Axis Robotic Arm	7
2.2.1 Selected Kinematic Architecture	7
2.2.2 Origin of the CAD Model and Adaptations	8
2.2.3 Mechanical Trade-off	9
2.3 Conclusion	9
3 Electronic Design	10
3.1 Introduction	10
Introduction	10

3.2	Component Selection	10
3.2.1	TowerPro MG90S Servomotors	10
3.2.2	4N35 Optocouplers	11
3.2.3	ESP32 Microcontroller	11
3.2.4	Summary of the Components Used	11
3.3	Electrical Schematic in KiCAD	12
3.3.1	Overview of the Schematic	12
3.3.2	Topology Selection	13
3.4	Printed Circuit Board Design	13
3.4.1	Single-Layer Routing	13
3.5	Conclusion	14
4	ROS 2 Software Architecture and Digital Twin	15
4.1	Introduction	15
	Introduction	15
4.2	Presentation of ROS 2	15
4.2.1	What is ROS 2?	15
4.2.2	Fundamental Concepts	16
4.2.3	RViz and Gazebo in a Nutshell	16
4.3	Robot Description: URDF	17
4.3.1	The URDF Format	17
4.3.2	From SolidWorks Model to URDF: Conversion Pipeline	17
4.3.3	URDF of the 4-Axis Arm	18
4.4	ROS 2 Node Architecture	19
4.4.1	Overview	19
4.4.2	Topics Used	20
4.4.3	Publishing a Joint Command – Code Excerpt	21
4.5	Launch File	21
4.5.1	Role of a Launch File	21
4.5.2	Structure of the Project Launch File	22
4.5.3	Usage	23
4.6	Visualization in RViz	23
4.7	Physical Simulation in Gazebo	23
4.7.1	Role of Gazebo in the Project	23
4.7.2	Simulation Limitations	24
4.8	Consequences of Open-Loop Control	25
4.9	Conclusion	25

5	ESP32 Embedded System and Integration	26
5.1	Introduction	26
	Introduction	26
5.2	Development Environment: PlatformIO and Arduino Core	26
5.2.1	Introduction to PlatformIO	26
5.3	Servomotor Control	27
5.3.1	Control Principle of Hobby Servos	27
5.3.2	Implementation in the Firmware	27
5.4	PC ↔ ESP32 Communication	28
5.4.1	Selected Serial Link	28
5.4.2	Frame Format	28
5.5	Tests and Results	29
5.6	Conclusion	29
	General Conclusion and Perspectives	30

List of Figures

2.1	Overview of the 4-axis robotic arm (SolidWorks model).	8
2.2	Detail of the end-effector at the end of the kinematic chain.	8
2.3	Photograph of the assembled robotic arm after servomotor installation.	9
3.1	Complete electrical schematic of the control board in KiCAD.	13
3.2	Routing view of the single-layer printed circuit board in KiCAD.	14
4.1	Visualization of the arm in RViz from the URDF descriptor.	19
4.2	Graph of ROS 2 nodes and topics of the project, captured with rqt_graph. . . .	20
4.3	Arm loaded in Gazebo with a simple scene environment.	24
5.1	Visual comparison between the physical robotic arm (left) and its RViz digital twin (right).	29

List of Tables

3.1	Main characteristics of the MG90S servomotor.	11
3.2	Components used on the control board.	12
4.1	Joints declared in the arm URDF (arduinoobot.urdf.xacro).	18
4.2	ROS 2 topics used by the architecture.	20

List of Abbreviations

Acronym	Definition
CAD	Computer-Aided Design
DOF	Degree Of Freedom
DDS	Data Distribution Service
ESP-IDF	Espressif IoT Development Framework
GPIO	General Purpose Input/Output
IDE	Integrated Development Environment
LCM	Layer Component Mounting
MCU	Microcontroller Unit
PCB	Printed Circuit Board
PWM	Pulse Width Modulation
ROS	Robot Operating System
ROS 2	Robot Operating System version 2
RViz	ROS Visualization
SDK	Software Development Kit
SDF	Simulation Description Format
SoC	System on Chip
UART	Universal Asynchronous Receiver Transmitter
URDF	Unified Robot Description Format
USB	Universal Serial Bus
XML	Extensible Markup Language
YAML	YAML Ain't Markup Language

General Introduction

Manipulation robotics today occupies a central place in both industry and academic research. From automotive assembly lines to biology laboratories, through automated kitchens and educational experiments, robotic arms have become the archetype of the robotic device capable of physically interacting with its environment. Their design brings together several disciplines: mechanics, electronics, embedded computing, and, more recently, digital simulation.

Alongside this diffusion, the notion of digital twin has established itself as a structuring paradigm. A digital twin is a faithful and synchronized virtual representation of a physical system, capable of reproducing its dynamic behavior for visualization, simulation, validation, or supervision purposes. In the particular case of a robotic arm, the digital twin allows validating a trajectory before executing it on the real system, prototyping control algorithms without hardware risk, and providing detailed visual feedback on the current state of the system.

It is within this dual perspective – mastery of a complete robotic system and implementation of a functional digital twin – that this Personal Professional Project (PPP) is situated. Our work focuses on the **design, realization, and control of a four-axis robotic arm**, coupled with a digital twin developed under Robot Operating System version 2 (ROS 2) with the Gazebo simulator. The objective is not solely to make an arm move: it is also to build, around this arm, the software ecosystem that makes it observable, simulable, and reprogrammable.

Problem Statement. How can one realize, with resources accessible to a student, a functional robotic platform whose physical movements are faithfully reproduced by a virtual model, and whose control relies on a modular and extensible software architecture?

Objectives. The project pursues five main objectives:

1. To have a 4-axis mechanical structure usable as an experimentation platform;
2. To design and manufacture a dedicated electronic board for servomotor control, integrating the necessary protections;
3. To realize an embedded firmware on an ESP32 microcontroller capable of driving the actuators and communicating with a host computer;
4. To build a digital twin of the arm in ROS 2, visualizable in RViz and simulable in Gazebo;
5. To establish a communication channel between the ROS 2 part and the embedded board so that virtual and physical movements are consistent.

Scope and Limitations. The selected servomotors are MG90S hobby servos, which do not have usable encoder feedback. The control of the axes is therefore fundamentally in open loop: the commanded position is assumed to be reached, without sensor verification. This limitation, acknowledged and documented in this report, conditions the entire software architecture.

Report Outline. This thesis is organized into five chapters. **Chapter 1** sets the general context, presents the notion of digital twin, and formalizes the specifications. **Chapter 2** describes the mechanical structure of the arm and the software working environment. **Chapter 3** details the electronic design: schematic in KiCAD, component selection, printed circuit board fabrication. **Chapter 4**, the central chapter, presents the ROS 2 architecture put in place and the digital twin in RViz and Gazebo. **Chapter 5** addresses the embedded part on the ESP32 and the PC–microcontroller communication, before presenting the tests carried out. A general conclusion draws up the assessment and opens perspectives.

Chapter 1

General Context and Specifications

1.1 Introduction

Before discussing the technical choices and implementation of the system, it is necessary to establish the framework within which the project takes place. This first chapter places the problem within the current landscape of manipulation robotics, introduces the concept of a digital twin applied to a robotic arm, and formalizes the specifications that will guide all the decisions presented in the following chapters.

1.2 General Context of Manipulation Robotics

A **manipulator arm** is an articulated mechanical system designed to position and orient, within a workspace, a terminal device called an end-effector. The first industrial robotic arms appeared in the 1960s (the Unimate on General Motors production lines [1]), and their use subsequently spread to virtually all manufacturing industries. More recently, smaller robotic arms—sometimes referred to as cobots or desktop arms—have made this technology accessible to universities, makers, and small and medium-sized enterprises.

From a kinematic perspective, a manipulator arm is characterized by its number of Degrees Of Freedom (DOF), that is, the number of independent joints composing its kinematic chain. A **4-axis** robotic arm allows its end-effector to be positioned within a three-dimensional subspace while controlling the orientation of the gripper along a relevant axis (typically vertical for pick-and-place operations). This number of degrees of freedom represents a good compromise between functional capabilities and implementation complexity, making it a widely adopted choice for educational projects and demonstrators.

1.3 The Digital Twin Concept

1.3.1 Definition and Characteristics

The concept of a **digital twin** was popularized by Michael Grieves in the early 2000s in the context of product lifecycle management [2]. In its most complete form, a digital twin is defined by three elements:

1. a **physical object** evolving in the real world;
2. a **virtual model** representing this object with a level of fidelity adapted to the intended use;
3. a **bidirectional synchronization link** between the two, propagating the state of the real world to the model and, conversely, decisions from the model to the real world.

1.3.2 Application to a Robotic Arm

In the case of a manipulator arm, the digital twin fulfills several complementary functions:

- **Visualization:** it provides a real-time graphical representation of the arm's joint configuration, allowing the operator to instantly understand the state of the system;
- **Preliminary simulation:** it allows a trajectory to be replayed before execution on the physical hardware in order to detect possible collisions, joint-limit violations, or singular configurations;
- **Software prototyping:** it serves as a test bench for developing algorithms (planning, control, perception) without monopolizing or risking damage to the hardware;
- **Monitoring and diagnostics:** in advanced configurations, it supplies health indicators and enables remote diagnostics.

Within the scope of this project, and given the absence of position feedback from the selected actuators, the digital twin operates mainly in predictive mode: it represents the command sent to the physical robotic arm. This limitation, already mentioned in the introduction, will be discussed in detail in Chapter 4.

1.4 Specifications

1.4.1 Functional Requirements

The system must:

- include an articulated mechanical structure with at least four independently controllable degrees of freedom;
- allow individual control of each of the four axes from a host computer;
- provide a synchronized graphical representation (digital twin) of the arm within a standard simulation and visualization environment;
- enable the coordinated execution of a joint-space trajectory, that is, the simultaneous or sequential movement of the four axes toward specified target positions;
- remain reusable, modifiable, and extensible for future work (end-effector replacement, sensor integration, addition of new algorithms).

1.4.2 Technical Constraints

- Host operating system: Ubuntu 24.04 LTS, chosen for its compatibility with ROS 2 Jazzy Jalisco;
- Robotics middleware: ROS 2 Jazzy [3];
- Visualization tool: RViz 2;
- Simulator: Gazebo (version distributed with ROS 2 Jazzy);
- Embedded microcontroller: ESP32 (Xtensa LX6 dual-core SoC), programmed using PlatformIO with the Arduino core;
- Actuators: TowerPro MG90S hobby servomotors [4], controlled via PWM;
- Electronic board: **single-layer PCB, handcrafted manufacturing**, with optocoupler isolation between control signals and the power stage;
- Mechanical CAD tool: SolidWorks (the CAD model of the arm comes from an open-source design and was adapted to the project's requirements);
- Electronic CAD tool: KiCAD [5].

References

- [1] IEEE Spectrum. “A digital twin implementation for mobile and collaborative robot scenarios for teaching robotics based on robot operating system,” Accessed: May 28, 2026. [Online]. Available: <https://ieeexplore.ieee.org/document/9766583>
- [2] M. Grieves, “Digital twin: Manufacturing excellence through virtual factory replication,” White Paper, 2014.
- [3] Open Robotics. “ROS 2 jazzy jalisco – documentation officielle,” Accessed: May 28, 2026. [Online]. Available: <https://docs.ros.org/en/jazzy/>
- [4] TowerPro. “MG90S micro servo datasheet.” Documentation constructeur du servomoteur, Accessed: May 28, 2026. [Online]. Available: <https://www.towerpro.com.tw/product/mg90s-3/>
- [5] KiCAD Project. “KiCAD EDA – documentation officielle,” Accessed: May 28, 2026. [Online]. Available: <https://www.kicad.org/help/documentation/>

Chapter 2

Mechanical Structure

2.1 Introduction

This chapter presents the mechanical structure selected for the four-axis robotic arm, acknowledging that this mechanical part was imported and adapted from an open-source design rather than being designed entirely from scratch.

2.2 Mechanical Structure of the 4-Axis Robotic Arm

2.2.1 Selected Kinematic Architecture

The selected robotic arm follows a **serial architecture with four revolute joints**, a configuration widely used in educational robotic arms and desktop arms. The kinematic chain is organized as follows, from the base to the end-effector:

1. **Axis 1 – Base Rotation** (vertical axis): provides the azimuthal orientation of the arm within its workspace;
2. **Axis 2 – Shoulder** (horizontal axis): controls the elevation of the first link;
3. **Axis 3 – Elbow** (horizontal axis): controls the angle between the two main links;
4. **Axis 4 – Wrist or Gripper** (depending on the installed end-effector): final joint, allowing the orientation of the terminal device.

This architecture is intentionally simple: with only four axes, the arm cannot reach any arbitrary pose in Cartesian space (at least one degree of freedom is missing for full orientation control), but it covers a sufficient workspace volume for the intended demonstrations while remaining compatible with simple analytical control.

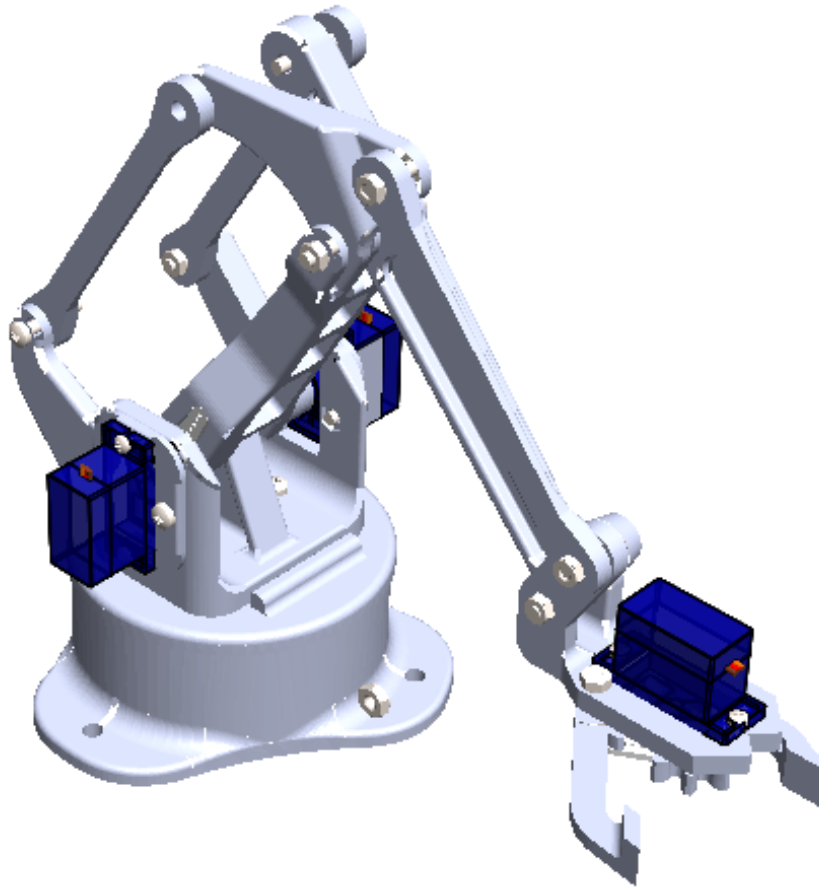


Figure 2.1. Overview of the 4-axis robotic arm (SolidWorks model).

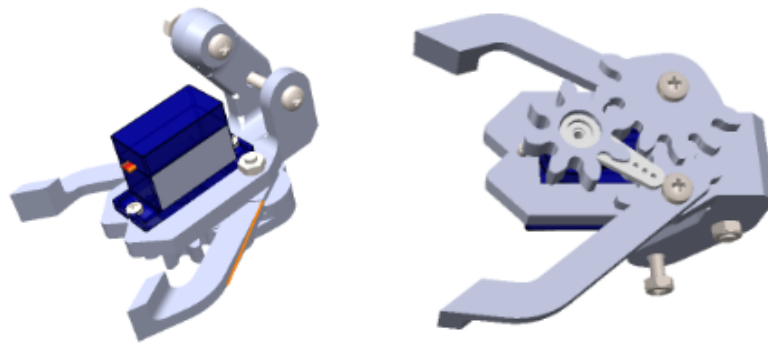


Figure 2.2. Detail of the end-effector at the end of the kinematic chain.

2.2.2 Origin of the CAD Model and Adaptations

Since the project focuses on the electronic, embedded, and software aspects, we chose not to design the entire mechanical structure from scratch. Instead, we imported an existing four-axis robotic arm model distributed under an open-source license and adapted it to our needs. This approach is deliberate and represents a conscious trade-off: it allows development efforts to be concentrated on the layers that provide the greatest educational value with respect to our training, while relying on a proven mechanical design.



Figure 2.3. Photograph of the assembled robotic arm after servomotor installation.

2.2.3 Mechanical Trade-off

The decision to use an imported CAD model offers clear advantages (time savings and the reliability of an already tested geometry) but also introduces certain limitations: we inherit the dimensional choices of the original designer, which are not necessarily optimal for our specific use case, and we forgo certain optimizations that would have required a complete redesign. This compromise is considered acceptable given the educational objectives of the project and is stated here for transparency.

2.3 Conclusion

This chapter presented the four-axis mechanical structure adapted from an open-source model. The next chapter addresses the electronic design itself: the control board schematic developed in KiCAD, component selection (particularly optocoupler isolation), and the handcrafted PCB manufacturing process.

Chapter 3

Electronic Design

3.1 Introduction

The electronics of the robotic arm have, at first glance, a single mission: translating the control signals generated by the microcontroller into articulated movements of the four servomotors. This apparent simplicity conceals several important practical constraints: hobby servomotors draw brief but high current peaks during startup, they generate significant electrical noise during operation, and their power ground must be isolated from the logic ground to protect the ESP32. This chapter presents the design choices adopted in response to these constraints, the schematic developed in KiCAD, the handcrafted PCB manufacturing process, and the trade-offs that guided each decision.

3.2 Component Selection

3.2.1 TowerPro MG90S Servomotors

The selected actuators are **TowerPro MG90S servomotors**, compact hobby servos equipped with a metal gear train. Their main characteristics [4] are summarized in Table 3.1.

The MG90S is a **pragmatic** choice, justified by:

- a **standard PWM control interface** natively compatible with the ESP32 timer/MCPWM peripherals;
- sufficient **torque** for the dimensions of the selected robotic arm.

The main limitation of the MG90S is the **absence of position feedback**: the servomotor receives an angular command (through the PWM pulse width) but provides no information about the actual position reached. This limitation is fundamental: it imposes **open-loop** control

and determines, as will be discussed in Chapter 4, the semantics of the digital twin (which reflects the command rather than the measurement).

Table 3.1. Main characteristics of the MG90S servomotor.

Characteristic	Typical Value
Supply voltage	4.8 – 6.0 V
Torque (at 4.8 V)	$\approx 1.8 \text{ kg}\cdot\text{cm}$
Torque (at 6.0 V)	$\approx 2.2 \text{ kg}\cdot\text{cm}$
Speed (at 4.8 V)	$0.10 \text{ s} / 60^\circ$
Gears	Metal
Control signal	PWM, 20 ms period, 1.0 – 2.0 ms pulse width
Angular range	$\approx 180^\circ$
Idle current	$\approx 10 \text{ mA}$
Operating current	100 – 250 mA
Startup current peak	up to 800 mA

3.2.2 4N35 Optocouplers

To protect the ESP32 and limit the propagation of electrical noise generated by the servomotors toward the microcontroller, we inserted a **4N35 optocoupler** on each PWM control line. This ideally guarantees complete galvanic isolation between the logic ground and the power ground.

3.2.3 ESP32 Microcontroller

The **ESP32** is a System on Chip (SoC) produced by Espressif Systems, integrating a dual-core Xtensa LX6 processor clocked at 240 MHz, embedded SRAM memory, and a rich set of peripherals (Wi-Fi, Bluetooth, ADC, UART, I2C, SPI, etc.). For servomotor control, it natively provides outputs capable of generating the required PWM signals, which can be easily exploited through the Servo library of the Arduino core. Details of its use are provided in Chapter 5.

3.2.4 Summary of the Components Used

The main components implemented on the control board are summarized in Table 3.2.

Table 3.2. Components used on the control board.

Component	KiCAD Reference	Role
4N35 Optocoupler	4N35	Galvanic isolation of a PWM control line
5-pin Connector	Conn_01x05	ESP32 signal input (4 PWM + ground)
3-pin Connectors	Conn_01x03	Output to each servomotor (signal, +V, GND)
220 Ω Resistor	R_220	Biasing of the internal LED of each optocoupler
Screw Terminal Block	Terminal_block	Power supply input
C1 10 μ F Capacitor	C_10uF	Power supply decoupling
Indicator LED	LED	Visual indication of power presence

3.3 Electrical Schematic in KiCAD

3.3.1 Overview of the Schematic

The board schematic is divided into three functional blocks:

1. **Power Supply Block:** power voltage input (typically 5–6 V), decoupling capacitors, and distribution to the servo connectors;
2. **Isolation Block:** four optocouplers, one per PWM line, with their biasing resistors;
3. **Connector Block:** ESP32 harness input connector (PWM signals + logic ground), four 3-pin standard servo connectors (signal, +V, power ground), and the main power supply terminal.

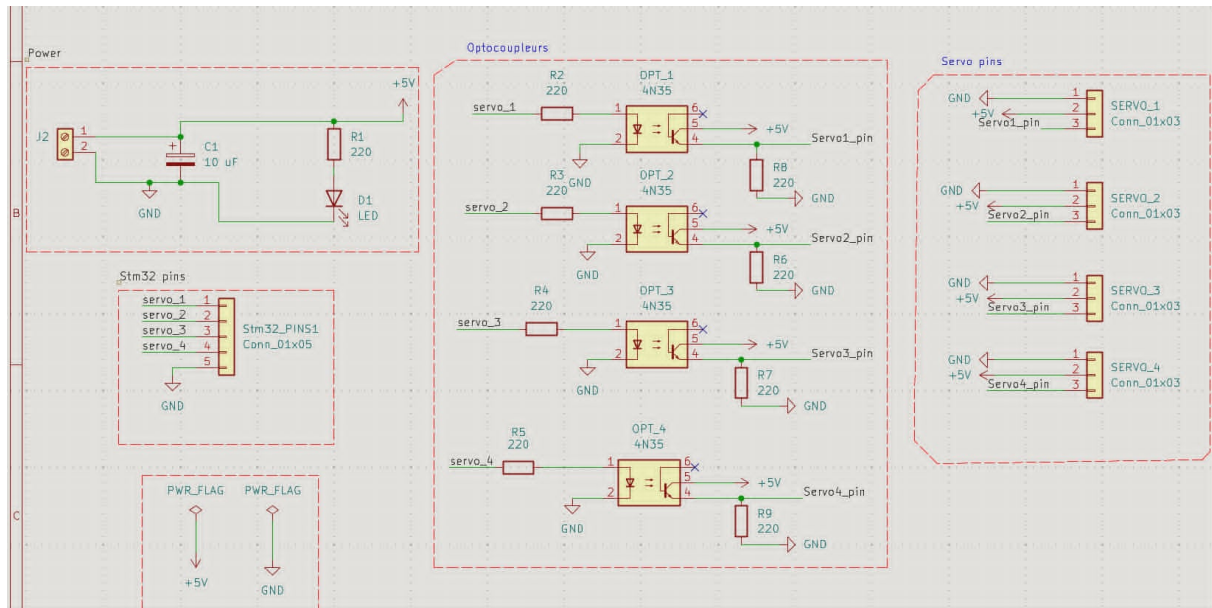


Figure 3.1. Complete electrical schematic of the control board in KiCAD.

3.3.2 Topology Selection

Two topologies were considered for interfacing the ESP32 with the servomotors:

- **Direct Connection:** the ESP32 directly drives the servomotors and shares their ground. Simple, but exposes the microcontroller to noise and possible voltage transients from the power side;
- **One Optocoupler per Line:** the selected solution. It provides the clearest separation between the logic domain and the power domain.

The choice of optocouplers is consistent with the project's philosophy: protecting the ESP32, which is more expensive and more complex to replace than the servos, takes priority over schematic simplicity.

3.4 Printed Circuit Board Design

3.4.1 Single-Layer Routing

The board is implemented as a **single-layer** (1-layer) PCB, meaning that all traces are routed on only one side of the board. This choice is dictated by the selected manufacturing process: **handcrafted fabrication** using either thermal transfer or UV exposure followed by chemical etching.

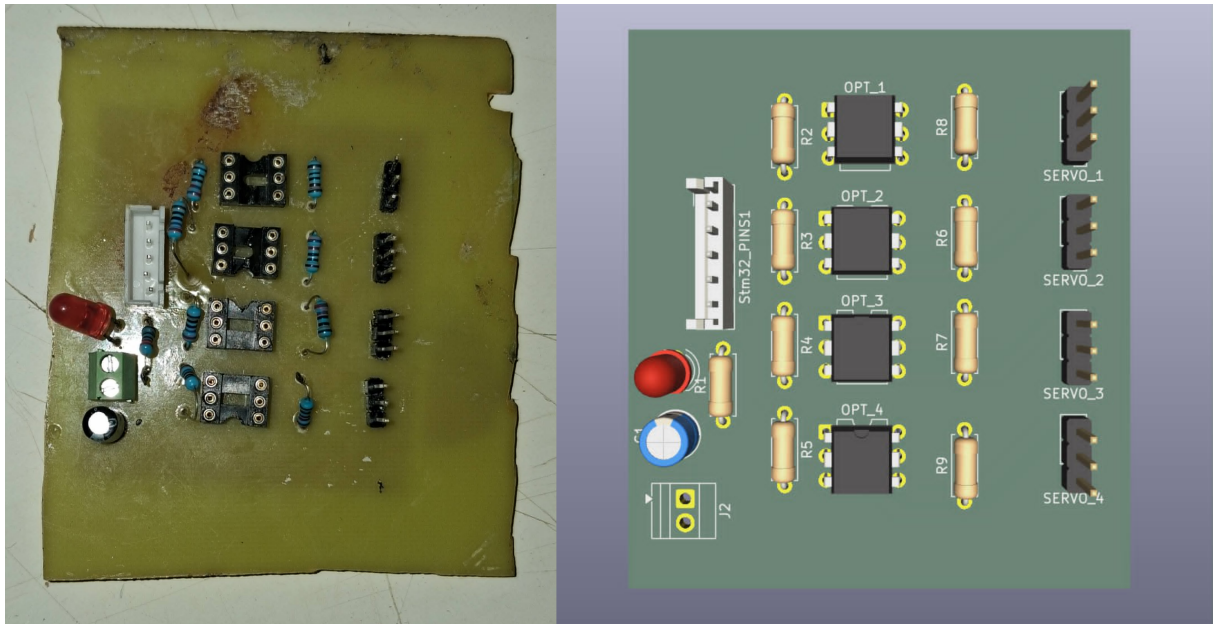


Figure 3.2. Routing view of the single-layer printed circuit board in KiCAD.

3.5 Conclusion

This chapter presented the electronic design of the project: selection of MG90S servomotors, isolation of the control lines using optocouplers, and handcrafted manufacturing of a single-layer PCB designed in KiCAD. The adopted trade-offs were motivated at each stage by the constraints of the student context (cost, schedule, available tools) and by the desire to protect the ESP32 from electrical noise and possible faults on the power side. The next chapter, which is central to this thesis, presents the ROS 2 software architecture and the digital twin that operates using this control board.

Chapter 4

ROS 2 Software Architecture and Digital Twin

4.1 Introduction

This chapter forms the software core of the project. It progressively introduces the ROS 2 ecosystem, assuming the reader does not necessarily have prior in-depth knowledge of it – unlike tools such as SolidWorks or KiCAD, which are part of the common curriculum of any engineering degree. Following this introduction, we describe the software architecture put in place: the URDF description of the arm, the nodes and topics used, visualization in RViz, and physical simulation in Gazebo. Python code excerpts illustrate the developed nodes.

4.2 Presentation of ROS 2

4.2.1 What is ROS 2?

ROS (Robot Operating System) is not, despite its name, an operating system: it is a **middleware** intended for the development of robotic applications. ROS provides a communication infrastructure between distributed programs (called nodes), a standard library of message types, a package management system, and a set of command-line tools.

ROS 2 is the second generation of ROS. Whereas ROS 1 relied on a centralized architecture with a master roscore, ROS 2 adopts a fully distributed architecture, based on the Data Distribution Service (DDS) standard. The main improvements of ROS 2 over ROS 1 are:

- the absence of a single master (increased resilience, no single point of failure);
- native support for **real-time** and **multi-machine** systems;
- better support for embedded systems (via micro-ROS);

- richer communication contracts (Quality of Service, or QoS).

The version used in this project is **ROS 2 Jazzy Jalisco**, distributed for Ubuntu 24.04 [3].

4.2.2 Fundamental Concepts

To understand the rest of this chapter, the reader needs to master a few key concepts of ROS 2:

- **Node**: an autonomous software process, generally dedicated to a single task (for example: reading a sensor, computing a setpoint, driving a motor);
- **Topic**: a named communication channel on which one or more nodes publish (publishers) and one or more others subscribe (subscribers). The model is asynchronous and anonymous (the publisher is unaware of who is listening);
- **Message**: the data structure exchanged on a topic, defined by a typed schema;
- **Service**: a synchronous request/response communication channel;
- **Action**: an asynchronous channel for long-duration goals (for example: “reach this position”), with periodic status feedback and the possibility of cancellation;
- **Package**: the distribution unit of ROS 2, containing one or more nodes, their dependencies, and their resources (URDF descriptors, launch files, etc.);
- **Launch file**: a Python script that describes how to simultaneously start a set of nodes with their parameters.

4.2.3 RViz and Gazebo in a Nutshell

RViz (ROS Visualization) is the standard graphical tool for visualizing, in 3D, the state of data circulating in ROS 2: robot poses, point clouds, frame transformations, markers, etc. RViz simulates nothing: it displays what the nodes publish.

Gazebo is, conversely, a physical simulator: it models rigid bodies, contacts, gravity, and sensors, and allows evaluating the behavior of a robot in a virtual environment. Gazebo communicates with ROS 2 via a bridge that exposes simulation topics as standard ROS 2 topics.

The complementarity between RViz and Gazebo is essential: RViz visualizes intentions (commands, trajectory plans) while Gazebo simulates physical reality. In our architecture, RViz will display the setpoint sent to the arm (whether real or simulated), and Gazebo will allow testing commands without hardware.

4.3 Robot Description: URDF

4.3.1 The URDF Format

URDF (Unified Robot Description Format) is an XML format used to describe the kinematics and geometry of a robot in ROS. A URDF file declares:

- **links** (rigid bodies) with their mass, center of inertia, visualization geometry, and collision geometry;
- **joints** (articulations) that connect links in pairs, with their type (revolute, prismatic, fixed), their axis, their angular limits, and their maximum velocities/efforts.

The same URDF file serves as a shared reference for RViz (for display), Gazebo (for physical simulation), and the kinematics nodes. This is one of the strengths of the ecosystem: a single source of truth describes the robot.

4.3.2 From SolidWorks Model to URDF: Conversion Pipeline

The URDF of the arm was not written from scratch: it was built from the CAD SolidWorks model adapted in chapter 2. The conversion pipeline follows three successive steps, illustrated in figure ??.

1. **Exporting SolidWorks parts (.sldprt) to STL format.** Each part of the assembly (basement, base_plate, forward_drive_arm, horizontal_arm, claw_support, fingers) is individually exported from SolidWorks in .STL (Stereolithography) format. STL is an open format universally supported by robotics tools: it describes the surface of the part as a triangle mesh, without parametric information. During export, we pay attention to two points: the **export frame** (which must coincide with the joint frame of the joint that carries the part in the URDF) and the **mesh density** (a mesh that is too fine burdens RViz without visible benefit);
2. **Organizing the meshes.** The generated .STL files are placed in the meshes/ folder of the ROS 2 robot description package (arduino_bot_description), with the name of the corresponding part (basement.STL, forward_drive_arm.STL, etc.). They are referenced from the URDF in the form `package://arduino_bot_description/meshes/<name>.STL`;
3. **Writing the URDF.** From the dimensions and axes measured in SolidWorks, we write the `arduino_bot.urdf.xacro` file by declaring one `<link>` per part (with its visual mesh and collision mesh pointing to the corresponding .STL) and one `<joint>` per articulation (with its type revolute, its axis, its limits, and the `<origin>` offset between the parent frame and

the child frame). The **xacro** (XML macros) format allows factoring out constants (PI, effort, velocity) and macros (default inertias), avoiding repetition from a raw URDF.

4.3.3 URDF of the 4-Axis Arm

Table 4.1 summarizes the joints declared in our URDF.

Table 4.1. Joints declared in the arm URDF (arduinobot.urdf.xacro).

Joint	Type	Axis	Limits	Parent → Child
joint_1	revolute	Z	$[-\pi/2, \pi/2]$	base_link → base_plate
joint_2	revolute	X	$[-\pi/2, \pi/2]$	base_plate → forward_drive_arm
joint_3	revolute	X	$[-\pi/2, \pi/2]$	forward_drive_arm → horizontal_arm
joint_4	revolute	Z	$[-\pi/2, 0]$	claw_support → gripper_right
joint_5	revolute (mimic)	Z	$[0, \pi/2]$	claw_support → gripper_left

Note: joint_5 is declared as a mimic of joint_4 (multiplier -1), ensuring symmetric closure of the gripper. The angular limits correspond to the capabilities of an MG90S (mechanical travel of approximately 180°) and will need to be adjusted to the values actually measured after calibration.

A simplified excerpt of the URDF file is given in listing 4.1.

```

1 <?xml version="1.0"?>
2 <robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="arduinobot">
3
4   <xacro:property name="PI"          value="3.14159265359"/>
5   <xacro:property name="effort"       value="30.0"/>
6   <xacro:property name="velocity"     value="10.0"/>
7
8   <link name="base_link">
9     <xacro:default_inertial mass="1.0"/>
10    <visual>
11      <geometry>
12        <mesh filename="package://arduinobot_description/meshes/
13          basement.STL"
14          scale="0.01 0.01 0.01"/>
15      </geometry>
16    </visual>
17  </link>
18
19  <joint name="joint_1" type="revolute">
20    <parent link="base_link"/>

```

```

20   <child link="base_plate"/>
21   <origin xyz="0 0 0.307"/>
22   <axis xyz="0 0 1"/>
23   <limit lower="-${PI/2}" upper="${PI/2}"
24         effort="${effort}" velocity="${velocity}"/>
25 </joint>
26
27 <!-- joint_2, joint_3, joint_4, joint_5 (mimic joint_4) -->
28 </robot>

```

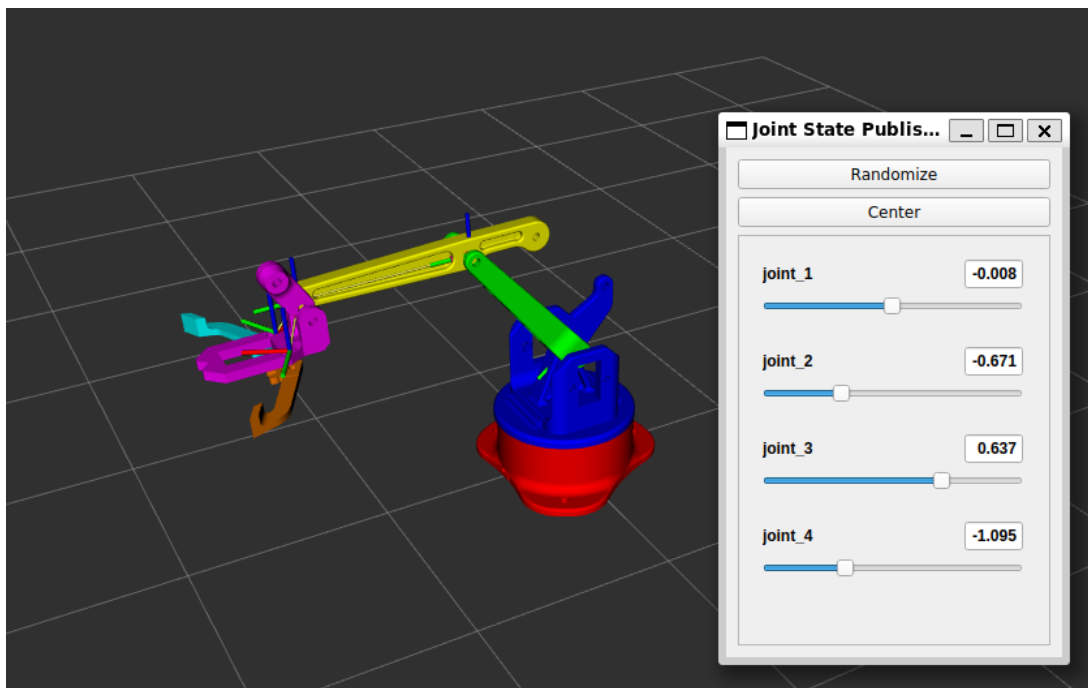
Listing 4.1: Excerpt from `arduinobot.urdf.xacro` (simplified).

Figure 4.1. Visualization of the arm in RViz from the URDF descriptor.

4.4 ROS 2 Node Architecture

4.4.1 Overview

The software architecture, as observed via `rqt_graph`, is organized around four main nodes:

- **robot_state_publisher** – standard ROS 2 node that, from the `/robot_description` parameter (the URDF expanded from `xacro`) and the current joint angles, publishes all transformations between frames on `/tf` and `/tf_static`;
- **joint_state_publisher_gui** – standard node activated by `display.launch.py`: it exposes a slider interface (one per joint) and publishes the corresponding angles on `/joint_states`, which animates the arm in RViz;

- **controller_manager / gz_ros_control** – in simulation mode, the gz_ros_control plugin embedded in Gazebo hosts a controller_manager that consumes /robot_description and executes position commands on the simulated joints;
- **ros_gz_bridge** – bidirectional bridge between Gazebo and ROS 2, configured in gazebo.launch.py to relay notably the simulation clock (/clock).

The communication diagram is represented in figure 4.2.

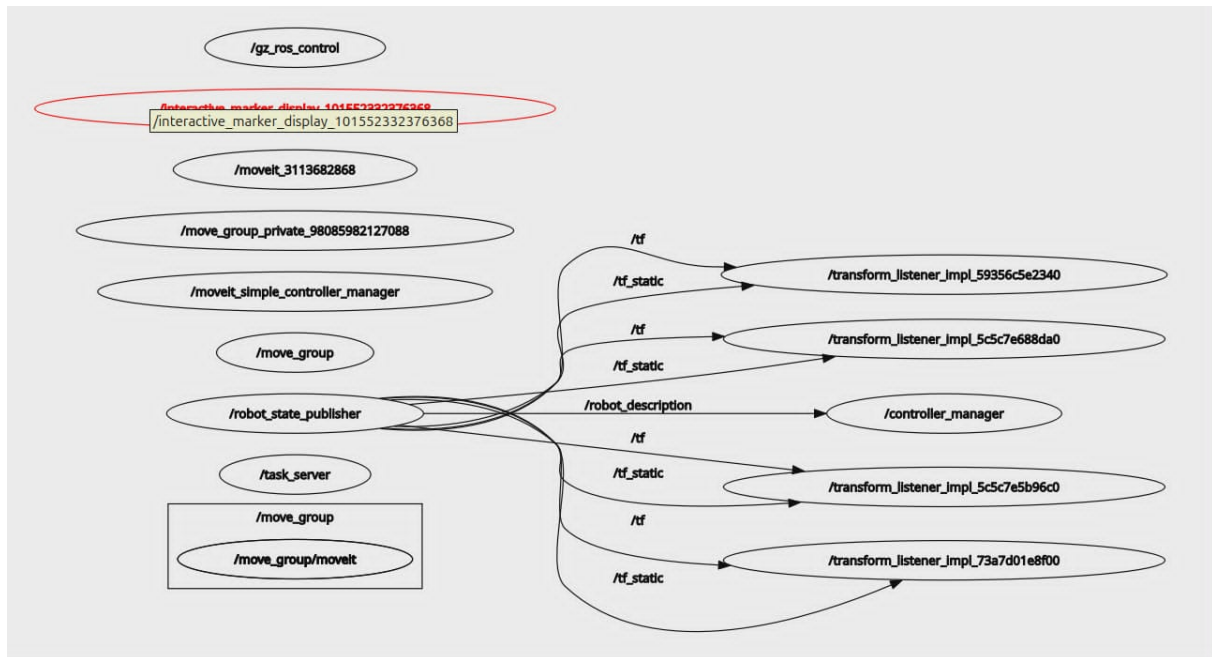


Figure 4.2. Graph of ROS 2 nodes and topics of the project, captured with rqt_graph.

4.4.2 Topics Used

Table 4.2 summarizes the topics used and their publishers/subscribers.

Table 4.2. ROS 2 topics used by the architecture.

Topic	Message Type	Publisher	Subscriber(s)
/joint_states	sensor_msgs/JointState	joint_state_publisher_gui	robot_state_publisher
/tf, /tf_static	tf2_msgs/TFMessage	robot_state_publisher	RViz, transform_listener_impl_*
/robot_description	std_msgs/String	robot_state_publisher	controller_manager (Gazebo)
/clock	roscpp_msgs/Clock	ros_gz_bridge (Gazebo)	all nodes with use_sim_time

4.4.3 Publishing a Joint Command – Code Excerpt

In display mode, the `joint_state_publisher_gui` interface is sufficient to animate the arm. For programmatic control (demonstration script, test scenario), it is common to directly publish a `JointState` message on `/joint_states` via the `rclpy` library. Listing 4.2 presents a minimal excerpt, using the actual joint names from the URDF.

```

1 import rclpy
2 from rclpy.node import Node
3 from sensor_msgs.msg import JointState
4
5 JOINT_NAMES = ["joint_1", "joint_2", "joint_3", "joint_4"]
6
7 class JointCommandPublisher(Node):
8     def __init__(self):
9         super().__init__("joint_command_publisher")
10        self.pub = self.create_publisher(JointState, "/joint_states",
11                                         10)
12        self.create_timer(0.1, self.tick)    # 10 Hz
13        self.target = [0.0, 0.0, 0.0, 0.0]
14
15    def tick(self):
16        msg = JointState()
17        msg.header.stamp = self.get_clock().now().to_msg()
18        msg.name = JOINT_NAMES
19        msg.position = self.target
20        self.pub.publish(msg)

```

Listing 4.2: Programmatic publication of a joint command (Python / `rclpy`).

This node periodically publishes (10 Hz) the joint command on `/joint_states`. Since `joint_5` is declared as a mimic of `joint_4` in the URDF, it does not need to be explicitly commanded: `robot_state_publisher` automatically derives its angle.

4.5 Launch File

4.5.1 Role of a Launch File

Manually starting each node (`robot_state_publisher`, `joint_state_publisher_gui`, `rviz2`, `ros_gz_sim`, `ros_gz_bridge`...) in a separate terminal would quickly become impractical. ROS 2 provides **launch files** for this purpose: these are Python scripts that declaratively describe all the nodes to start, their parameters, their topic remappings, and the associated resources (URDF, RViz configuration, etc.). A single call to `ros2 launch` is then sufficient to instantiate the complete stack.

4.5.2 Structure of the Project Launch File

The launch file used to start the digital twin is named `display.launch.py` and placed in the `launch/` folder of the `arduinobot_description` package. It performs three operations:

1. **Reading the xacro:** the `arduinobot.urdf.xacro` file is declared as a launch argument (model) and expanded via the `xacro` command, then injected into the `robot_description` parameter;
2. **Starting the standard nodes:** `robot_state_publisher` (which uses `robot_description` to publish `/tf` and `/tf_static`) and `joint_state_publisher_gui` (which produces the `/joint_states` topic via interactive sliders);
3. **Starting RViz:** with the pre-saved configuration file `display.rviz` that defines the panels to display, the background colors, and the visualized frames.

Listing 4.3 presents a simplified version, faithful to the actual file.

```

1 from launch import LaunchDescription
2 from launch_ros.actions import Node
3 from launch_ros.parameter_descriptions import ParameterValue
4 from launch.actions import DeclareLaunchArgument
5 from launch.substitutions import Command, LaunchConfiguration
6 import os
7 from ament_index_python.packages import get_package_share_directory
8
9 def generate_launch_description():
10     pkg = get_package_share_directory("arduinobot_description")
11
12     model_arg = DeclareLaunchArgument(
13         name="model",
14         default_value=os.path.join(pkg, "urdf", "arduinobot.urdf.
15             xacro"))
16
17     robot_description = ParameterValue(
18         Command(["xacro ", LaunchConfiguration("model")]))
19
20     return LaunchDescription([
21         model_arg,
22         Node(package="robot_state_publisher",
23             executable="robot_state_publisher",
24             parameters=[{"robot_description": robot_description}]),
25         Node(package="joint_state_publisher_gui",
26             executable="joint_state_publisher_gui"),
27         Node(package="rviz2", executable="rviz2",

```

```
27         arguments=["-d", os.path.join(pkg, "rviz", "display.rviz  
28         ")]),  
    ])
```

Listing 4.3: Simplified excerpt from `launch/display.launch.py`.

4.5.3 Usage

Execution is then done with a single command:

```
ros2 launch arduinobot_description display.launch.py
```

Listing 4.4: Starting the visualization stack with a single command.

A second launch file, `gazebo.launch.py`, starts the simulation: it expands the same `xacro`, sets the `GZ_SIM_RESOURCE_PATH` environment, includes `ros_gz_sim` with an empty world, instantiates the robot via `ros_gz_sim/create`, and relays the `/clock` via `ros_gz_bridge`.

This single command replaces several terminals and ensures that all nodes are started with the same parameters and in the same order at every execution: this is an essential point for the **reproducibility** required by the specifications.

4.6 Visualization in RViz

RViz displays the arm in real time in the configuration defined by the `/joint_states` topic, relying on the URDF and the transformations published by `robot_state_publisher`. Concretely:

1. the `joint_state_publisher_gui` node exposes one slider per joint (`joint_1` to `joint_4`) and publishes the angles on `/joint_states`;
2. `robot_state_publisher` converts these angles into transformations on `/tf` and `/tf_static`, applying the mimic rule for `joint_5`;
3. **RViz** reads `/tf` and the meshes declared in the URDF (`basement.STL`, `forward_drive_arm.STL`, etc.) and redraws the arm.

4.7 Physical Simulation in Gazebo

4.7.1 Role of Gazebo in the Project

Gazebo, in our architecture, plays two complementary roles:

- **Hardware-free test bench:** it allows executing ROS 2 nodes (notably the `controller_manager` hosted by `gz_ros_control`) without needing to connect the real arm, which accelerates development;
- **Prior validation:** it allows visualizing, in a simulated physical environment (gravity, contacts), the expected behavior before executing it on hardware.

The arm is loaded in Gazebo from the same URDF used by RViz, which ensures consistency between the two views.

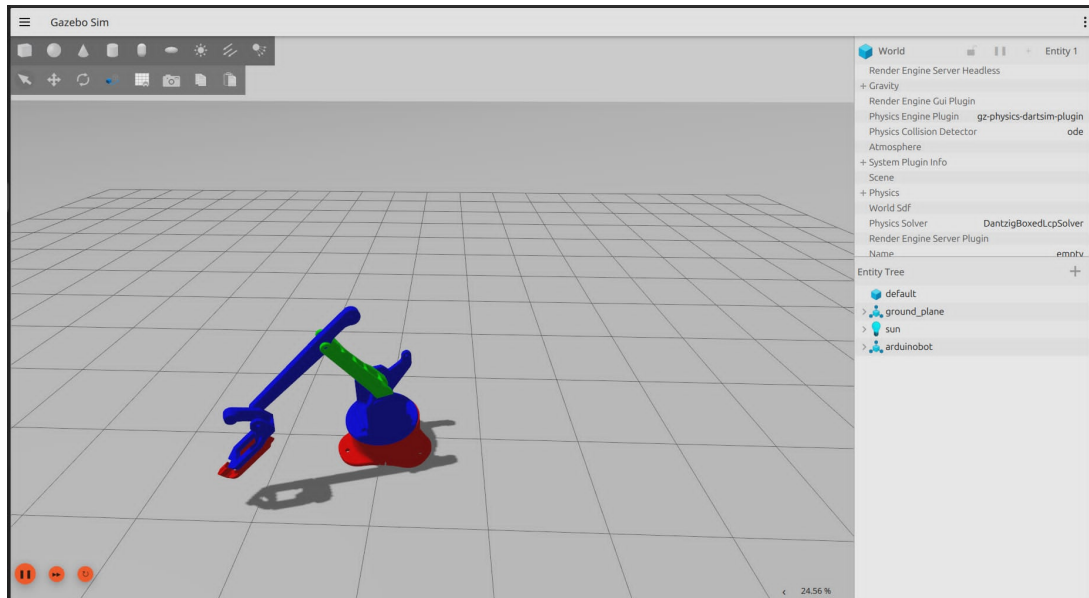


Figure 4.3. Arm loaded in Gazebo with a simple scene environment.

4.7.2 Simulation Limitations

It is important to be transparent about the limitations of the Gazebo simulation in the context of this project:

- Gazebo faithfully simulates **kinematics** (positions, velocities) but the **fine dynamics** of hobby servomotors (mechanical backlash, hysteresis, non-linear response time) are not modeled by default;
- the simulator ignores **electrical noise** and any potential **glitches** in the physical connection;
- the absence of position feedback from the real arm makes it impossible to inject the measured position back into Gazebo, which therefore remains autonomous.

4.8 Consequences of Open-Loop Control

As stated from the introduction, the arm is driven in **open loop**: the ESP32 sends a PWM pulse width to the servomotor, which attempts to reach that position, but no feedback is measured. On the software side, this translates to:

- the `/joint_states` topic reflects the command (from `joint_state_publisher_gui` or an application publisher) and not the measurement;
- RViz and Gazebo display an idealized representation of the arm, which may silently diverge from reality (insufficient torque, undeclared mechanical stop, snagging);
- it is not possible to automatically detect a positioning fault.

4.9 Conclusion

This chapter has presented the software architecture of the project: a single xacro descriptor (`arduinobot.urdf.xacro`) shared between RViz and Gazebo, two launch files that respectively start the visualization stack (`display.launch.py`) and the physical simulation (`gazebo.launch.py`), and the use of standard tools from the ecosystem (`robot_state_publisher`, `joint_state_publisher_gui`, `ros_gz_bridge`). The digital twin produced is consistent with the specifications, subject to the limitation imposed by open-loop control. The final chapter details the embedded part e...

Chapter 5

ESP32 Embedded System and Integration

5.1 Introduction

This final technical chapter presents the embedded part of the project: the firmware running on the ESP32, the generation of control signals intended for the servomotors, and the communication channel between the host computer and the microcontroller. In accordance with the project objectives, which are focused on robotics and ROS 2 software architecture, the embedded part was deliberately kept simple: the goal is not to perform advanced embedded engineering, but rather to quickly obtain correct execution of commands sent from the PC.

5.2 Development Environment: PlatformIO and Arduino Core

5.2.1 Introduction to PlatformIO

PlatformIO is a cross-platform development environment for embedded systems, provided as an extension for editors such as Visual Studio Code. It offers:

- automatic management of compilation **toolchains** for a wide range of targets (ESP32, STM32, AVR, etc.);
- **library** management through a single configuration file (`platformio.ini`);
- integrated commands for compilation, flashing, and serial monitoring.

PlatformIO was used together with the Arduino Core.

5.3 Servomotor Control

5.3.1 Control Principle of Hobby Servos

A hobby servomotor such as the MG90S expects a PWM signal with a fixed period (typically 20 ms, i.e., 50 Hz) whose pulse width encodes the angular command. In practice, on the ESP32 using the Arduino Core, the Servo library fully encapsulates this mechanism: it is sufficient to provide an angle in degrees, and the library generates the corresponding signal on the selected pin.

5.3.2 Implementation in the Firmware

The embedded firmware consists of only a few dozen lines. Listing 5.1 presents its complete structure.

```
1 #include <Arduino.h>
2 #include <ESP32Servo.h>
3
4 Servo servo_base;
5 Servo servo_shoulder;
6 Servo servo_elbow;
7 Servo servo_wrist;
8
9 const int PIN_BASE      = 13;
10 const int PIN_SHOULDER = 12;
11 const int PIN_ELBOW    = 14;
12 const int PIN_WRIST    = 27;
13
14 void setup() {
15     Serial.begin(115200);
16     servo_base.attach(PIN_BASE);
17     servo_shoulder.attach(PIN_SHOULDER);
18     servo_elbow.attach(PIN_ELBOW);
19     servo_wrist.attach(PIN_WRIST);
20 }
21
22 void loop() {
23     if (Serial.available()) {
24         String line = Serial.readStringUntil('\n');
25         int a1, a2, a3, a4;
26         if (sscanf(line.c_str(), "%d %d %d %d", &a1, &a2, &a3, &a4) ==
27             4) {
28             servo_base.write(a1);
29             servo_shoulder.write(a2);
```

```

29     servo_elbow.write(a3);
30     servo_wrist.write(a4);
31 }
32 }
33 }

```

Listing 5.1: Simplified embedded firmware: control of the four servos from commands received through the serial link.

The firmware therefore consists simply of: initializing the four Servo objects in `setup()`, followed by a loop that reads a frame from the serial port, decodes four integer angles, and applies them to the servomotors using `write()`. No FreeRTOS tasks are explicitly created, and no duty-cycle calculations are performed by the user: the Servo library handles everything.

5.4 PC ↔ ESP32 Communication

5.4.1 Selected Serial Link

Communication between the host computer (running the ROS 2 stack) and the ESP32 (controlling the servomotors) relies on the **USB serial connection** provided by the USB-to-serial converter integrated into the ESP32-DevKit board. This choice is motivated by:

- **simplicity**: no additional protocol needs to be implemented on the ESP32 side (the serial port is directly exposed by the Arduino Core);
- **reliability**: no packet loss caused by an unstable Wi-Fi network;
- **sufficient bandwidth**: 115 200 baud easily supports several dozen updates per second for four angles.

5.4.2 Frame Format

The selected frame format is deliberately minimalist: an ASCII line beginning with the character `,`, followed by four integer angles (in degrees) separated by spaces, and terminated by a newline character:

```
, <theta1> <theta2> <theta3> <theta4>\n
```

Listing 5.2: Joint command frame format sent from the PC to the ESP32.

On the PC side, the `serial_bridge` node (introduced in Chapter 4) subscribes to `/joint_states_cmd`, converts the angles into degrees, and transmits the corresponding frame over the serial connection at 115 200 baud.

5.5 Tests and Results

This test consists of executing the same sequence of commands simultaneously on the real robotic arm and on the arm simulated in Gazebo, and visually comparing the two. The kinematic consistency is very good; the remaining differences mainly arise from variations between individual MG90S servomotors, mechanical backlash in the joints, and the granularity of integer-degree control.

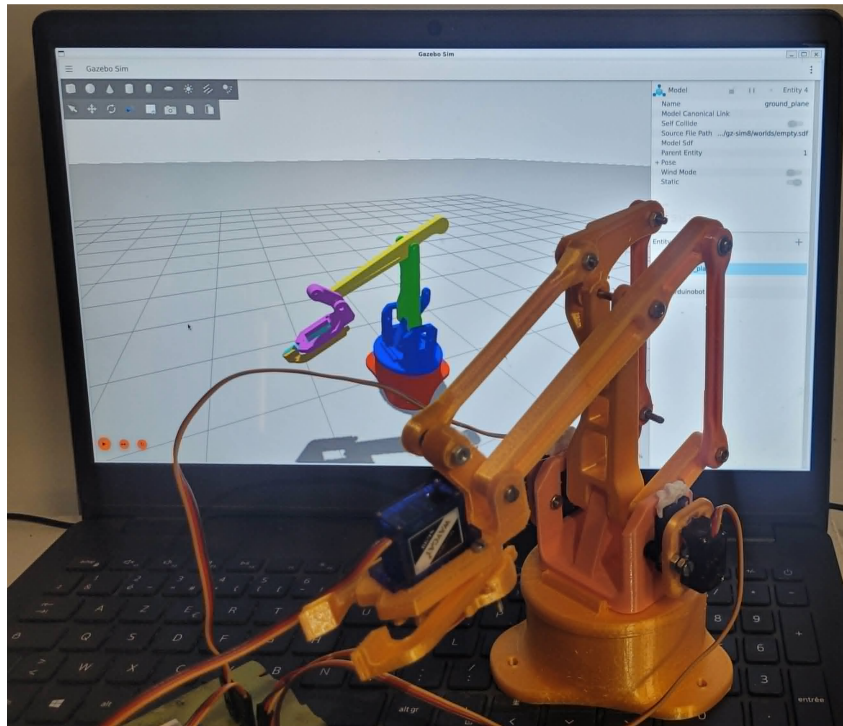


Figure 5.1. Visual comparison between the physical robotic arm (left) and its RViz digital twin (right).

5.6 Conclusion

This chapter presented the embedded part of the system: a short and readable ESP32 firmware written using PlatformIO and the Arduino Core, relying on the Servo library to control the four joints from a simple serial frame. The conducted tests confirm that the complete chain, from the ROS 2 node to the physical movement of the robotic arm, operates end-to-end. The observed differences between the real arm and its digital twin are consistent with what can be expected from an open-loop control system using hobby servomotors. The general conclusion of the report summarizes the overall results and proposes possible improvements.

General Conclusion and Perspectives

Project Summary

At the end of this Personal Professional Project, we have successfully completed the design and realization of a four-axis robotic arm coupled with a digital twin under ROS 2. The resulting prototype is fully functional: a hand-crafted single-layer control board provides optocoupler isolation and power distribution to the four MG90S servomotors, and the ROS 2 architecture put in place allows controlling the arm and visualizing its digital twin in RViz from a shared URDF descriptor.

Identified Limitations

Several limitations of the realized system must be transparently highlighted:

- the **absence of encoder feedback** on the MG90S servomotors imposes open-loop control. The digital twin therefore reflects the command and not the actually reached position;
- the **torque capacity** of the MG90S limits the manipulable payload as well as the acceptable speed at the end of the arm;
- the **update frequency** of the PC–ESP32 communication was sized for the needs of the project, but has not been optimized for high-dynamics trajectories;
- the **Gazebo simulation environment** reproduces kinematics, but the fine dynamics of hobby servomotors (mechanical backlash, hysteresis, overshoot) are not modeled.

Perspectives

Several improvement directions can be considered:

- **Closed-loop operation.** Replacing the MG90S servos with servos equipped with angle feedback (or adding external encoders) would allow closing the loop and truly servo-controlling the arm. The digital twin could then be fed by the measured position rather than the command;
- **Inverse kinematics and trajectory planning.** Integrating MoveIt 2 into the ROS 2 stack would allow specifying goals in Cartesian space rather than joint space and benefit from planning with obstacle avoidance;
- **PCB evolution.** Moving to a mass-produced double-layer PCB would improve electrical reliability, allow clean ground planes, and facilitate the addition of protections (TVS diodes, resettable fuses);
- **Real-time communication.** Replacing the current serial link with micro-ROS on the ESP32, or with an Ethernet link, would open the way to deeper integration into the DDS ecosystem of ROS 2;
- **Vision and perception.** Adding a camera and integrating a vision node would transform the arm into a true autonomous manipulator capable of grasping visually identified objects.

Beyond the technical object realized, this project has been a formative experience for us: it taught us to reconcile heterogeneous constraints, to rigorously document our choices, and to reason in terms of systems rather than isolated components. These skills seem to us transferable to any future engineering project.